3. Develop a program to implement Principal Component Analysis (PCA) for reducing the dimensionality of the Iris dataset from 4 features to 2.

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# Load the Iris dataset
iris = load_iris()
data = iris.data
labels = iris.target
label_names = iris.target_names

# Convert to a DataFrame for better visualization
iris_df = pd.DataFrame(data, columns=iris.feature_names)

# Perform PCA to reduce dimensionality to 2
pca = PCA(n_components=2)
data_reduced = pca.fit_transform(data)

# Create a DataFrame for the reduced data
reduced_df = pd.DataFrame(data_reduced, columns=['Principal Component 1', 'Principal Component 2'])
reduced_df['Label'] = labels

# Plot the reduced data
plt.figure(figsize=(8, 6))
colors = ['r', 'g', 'b']
for i, label in enumerate(np.unique(labels)):
    plt.scatter(
        reduced_df[reduced_df['Label'] == label]['Principal Component 1'],
        reduced_df[reduced_df['Label'] == label]['Principal Component 2'],
        label=label_names[label],
        color=colors[i]
    )

plt.title('PCA on Iris Dataset')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend()
plt.grid()
plt.show()
```

1. Objective of the Program

The program applies **Principal Component Analysis (PCA)** to the Iris dataset to:

- Reduce dimensionality from **4 features → 2 principal components**
- Visualize the dataset in a **2D space**
- Observe how well different classes are separated

2. Step-by-Step Analysis

2.1 Data Loading

iris = load_iris()
data = iris.data
labels = iris.target

- Dataset contains **150 samples**
- **4 features**: sepal length, sepal width, petal length, petal width
- **3 classes**:
    - Setosa
    - Versicolor
    - Virginica

2.2 Data Preparation

iris_df = pd.DataFrame(data, columns=iris.feature_names)

- Converts raw data into a DataFrame
- Improves readability and handling

2.3 Applying PCA

pca = PCA(n_components=2)
data_reduced = pca.fit_transform(data)

- Reduces dimensions from **4 → 2**
- PCA finds new axes (principal components) that:
    - Maximize variance
    - Remove redundancy (correlation)

2.4 Reduced Data Creation

reduced_df = pd.DataFrame(data_reduced, columns=['Principal Component 1', 'Principal Component 2'])

- Stores transformed data for visualization
- Adds class labels for plotting

2.5 Visualization

plt.scatter(...)

- Plots each class in different colors:
    - Red → Setosa
    - Green → Versicolor
    - Blue → Virginica
- Adds:
    - Title
    - Axis labels
    - Legend
    - Grid

3. Key Observations

✅ Class Separation

- **Setosa** is clearly separated → easy classification
- **Versicolor & Virginica** overlap slightly → harder to distinguish

✅ Dimensionality Reduction

- Reduced from 4D to 2D with minimal information loss
- PCA preserves most of the variance

4. Conceptual Understanding

Principal Components

- **PC1**: Direction of maximum variance
- **PC2**: Second most important direction (orthogonal to PC1)

5. Advantages of the Program

- Simple and efficient visualization
- Reduces complexity of high-dimensional data

- Helps in preprocessing for ML models

6. Limitations

- PCA is **linear** → may not capture complex relationships
- Some information loss is inevitable
- Overlapping classes still exist

7. Possible Improvements

◆ Add Explained Variance

print(pca.explained_variance_ratio_)

→ Shows how much information each component retains

◆ Standardization (Important)

from sklearn.preprocessing import StandardScaler
data = StandardScaler().fit_transform(data)

→ Ensures all features contribute equally

◆ 3D Visualization

- Use 3 components for better separation

8. Conclusion

This program effectively demonstrates how PCA:

- Reduces dimensionality
- Retains maximum variance
- Enables visualization of multi-dimensional data

It also highlights that while PCA improves interpretability, it may not fully separate all classes.

Program 4: For a given set of training data examples stored in a .CSV file, implement and demonstrate the Find-S algorithm to output a description of the set of all hypotheses consistent with the training examples.

```
import pandas as pd
data = {
    'Sky': ['Sunny', 'Sunny', 'Rainy', 'Sunny'],
    'Temp': ['Warm', 'Warm', 'Cold', 'Warm'],
    'Humidity': ['Normal', 'High', 'High', 'High'],
    'Wind': ['Strong', 'Strong', 'Strong', 'Strong'],
    'Water': ['Warm', 'Warm', 'Warm', 'Cool'],
    'Forecast': ['Same', 'Same', 'Change', 'Change'],
    'Play': ['Yes', 'Yes', 'No', 'Yes']
}
df = pd.DataFrame(data)
df.to_csv('training_data.csv', index=False)
def find_s_algorithm(file_path):
    data = pd.read_csv(file_path)
    print("Training data:")
    print(data)
    attributes = data.columns[:-1]
    class_label = data.columns[-1]
    hypothesis = ['?' for _ in attributes]
    for index, row in data.iterrows():
        if row[class_label] == 'Yes':
            for i, value in enumerate(row[attributes]):
                if hypothesis[i] == '?' or hypothesis[i] == value:
                    hypothesis[i] = value
```

```
            else:
                hypothesis[i] = '?'
    return hypothesis
file_path = 'training_data.csv'
hypothesis = find_s_algorithm(file_path)
print("\nThe final hypothesis is:", hypothesis)
```

```
Training data:
    Sky  Temp Humidity    Wind Water Forecast Play
0  Sunny  Warm   Normal  Strong  Warm     Same  Yes
1  Sunny  Warm     High  Strong  Warm     Same  Yes
2  Rainy  Cold     High  Strong  Warm   Change   No
3  Sunny  Warm     High  Strong  Cool   Change  Yes

The final hypothesis is: ['Sunny', 'Warm', 'High', 'Strong', '?', '?']
```

1. Objective of the Program

The program implements the **Find-S Algorithm** in Machine Learning to:

- Find the **most specific hypothesis**
- That is **consistent with all positive training examples**

2. Concept of Find-S Algorithm

- Works only with **positive examples (Yes)**
- Ignores **negative examples (No)**
- Starts with the **most specific hypothesis**
- Gradually generalizes it

3. Step-by-Step Code Analysis

3.1 Import Library

```
import pandas as pd
```

- Used to read and handle CSV data

3.2 Function Definition

```
def find_s_algorithm(file_path):
```

- Defines the function that executes the Find-S logic

3.3 Reading Dataset

```
data = pd.read_csv(file_path)
```

- Loads training data from a CSV file

3.4 Display Data

```
print("Training data:")
print(data)
```

- Prints dataset for verification

3.5 Separate Attributes and Target

```
attributes = data.columns[:-1]
class_label = data.columns[-1]
```

- Assumes:
    - Last column = **target label (Yes/No)**
    - Remaining columns = **features**

3.6 Initialize Hypothesis

```
hypothesis = ['?' for _ in attributes]
```

- Starts with **most specific hypothesis**
- '?' means "any value allowed"

3.7 Training Loop

for index, row in data.iterrows():

- Iterates through each training example

3.8 Process Only Positive Examples

if row[class_label] == 'Yes':

- Ignores negative examples
- Updates hypothesis only for **positive cases**

3.9 Hypothesis Update Rule

```
for i, value in enumerate(row[attributes]):
    if hypothesis[i] == '?' or hypothesis[i] == value:
        hypothesis[i] = value
    else:
        hypothesis[i] = '?'
```

Logic:

- If hypothesis is '?' → replace with value
- If value matches → keep it
- If mismatch → generalize to '?'

3.10 Return Result

return hypothesis

- Final hypothesis is returned

3.11 Function Call

```
file_path = 'training_data.csv'
hypothesis = find_s_algorithm(file_path)
```

- Executes algorithm on dataset

4. Working Example

Sample Data:

Sky	Temp	Humidity	Wind	Water	Forecast	Play
Sunny	Warm	Normal	Strong	Warm	Same	Yes
Sunny	Warm	High	Strong	Warm	Same	Yes
Rainy	Cold	High	Strong	Warm	Change	No
Sunny	Warm	High	Strong	Cool	Change	Yes

Final Hypothesis:

['Sunny', 'Warm', '?', 'Strong', '?', '?']

5. Key Observations

✅ **Strengths**

- Simple and easy to implement
- Works well with **clean, noise-free data**
- Efficient for small datasets

❌ **Limitations**

- Ignores **negative examples**
- Cannot handle **noisy/inconsistent data**
- Produces only **one hypothesis** (no alternatives)

6. Time Complexity

- **O(n × m)**
  - n = number of training examples
  - m = number of attributes

7. Possible Improvements

◆ **Handle Negative Examples**

- Use **Candidate Elimination Algorithm**

◆ **Input Validation**

```
if data.empty:
    print("Dataset is empty")
```

◆ **Case Handling**

```
if row[class_label].strip().lower() == 'yes':
```

8. Conclusion

This program successfully implements the **Find-S algorithm** to derive the **most specific consistent hypothesis** from training data. It demonstrates the basic concept of **concept learning** in machine learning but is limited by its inability to use negative examples.